Low-Level Design (LLD)

Automated HVAC Duct Detection and Annotation Web Application

1. LLD Objective

The purpose of this LLD is to translate the HLD into implementable engineering components for a **web application** that:

1. accepts HVAC drawing PDFs through a browser
2. creates a processing job
3. runs asynchronous duct detection and annotation
4. stores outputs and metadata
5. returns annotated preview and downloadable results

The sample indicates the output is a **visual markup over the original mechanical plan**, so the LLD emphasizes rendering, coordinate mapping, label association, and result serving rather than only raw extraction.

2. Suggested Tech Stack

Frontend

- Next.js or React
- TypeScript
- TanStack Query or SWR
- PDF/image viewer with zoom/pan

Backend

- FastAPI preferred
- Python for tight integration with PDF/CV pipeline
- Pydantic for request/response schemas
- SQLAlchemy for DB ORM

Async Processing

- Celery or RQ
- Redis as broker/cache

Processing Engine

- PyMuPDF
- pdfplumber
- OpenCV
- Shapely
- NumPy
- Pandas

Data/Infra

- PostgreSQL
- S3-compatible object storage
- Docker

3. LLD Scope

This LLD covers:

- frontend pages and UI interactions
- backend API routes and handlers
- queue payloads
- worker lifecycle
- PDF processing modules
- annotation generation modules
- database tables
- storage conventions
- failure handling
- configuration model

4. Component Breakdown

At LLD level, the system is split into the following concrete components:

1. **Frontend UI**
2. **Auth/Session Layer**
3. **Upload Service**
4. **Job Service**
5. **Result Service**
6. **Admin/Monitoring Service**
7. **Queue Producer**
8. **Worker Runtime**
9. **Processing Pipeline Modules**
10. **Persistence Layer**
11. **Object Storage Layer**

5. Frontend LLD

5.1 Pages

Upload Page

Responsibilities:

- select PDF
- validate extension/size
- upload file
- optional page selection
- trigger processing job

Job List Page

Responsibilities:

- show recent jobs
- show statuses
- allow reopen/download

Job Detail Page

Responsibilities:

- show file metadata
- show progress/status
- show error state if failed
- show result preview if completed

Result Viewer

Responsibilities:

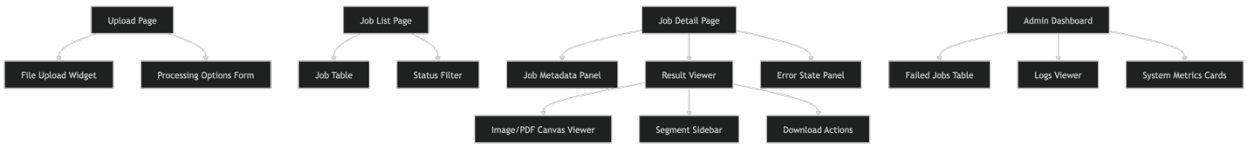
- image/PDF preview
- zoom/pan
- overlay metadata panel
- segment list sidebar

Admin Dashboard

Responsibilities:

- failed jobs
- worker errors
- logs
- system health summary

5.2 Frontend Component Diagram



6. Backend Service LLD

6.1 Service Modules

`upload_service`

Responsibilities:

- validate file
- create storage object key
- upload original file
- create DB file record

`job_service`

Responsibilities:

- create job
- validate page selection
- enqueue worker payload
- update status

`result_service`

Responsibilities:

- fetch outputs
- build preview response
- build segment summaries
- generate download URLs

`admin_service`

Responsibilities:

- fetch failed jobs
- fetch logs
- inspect worker outcomes

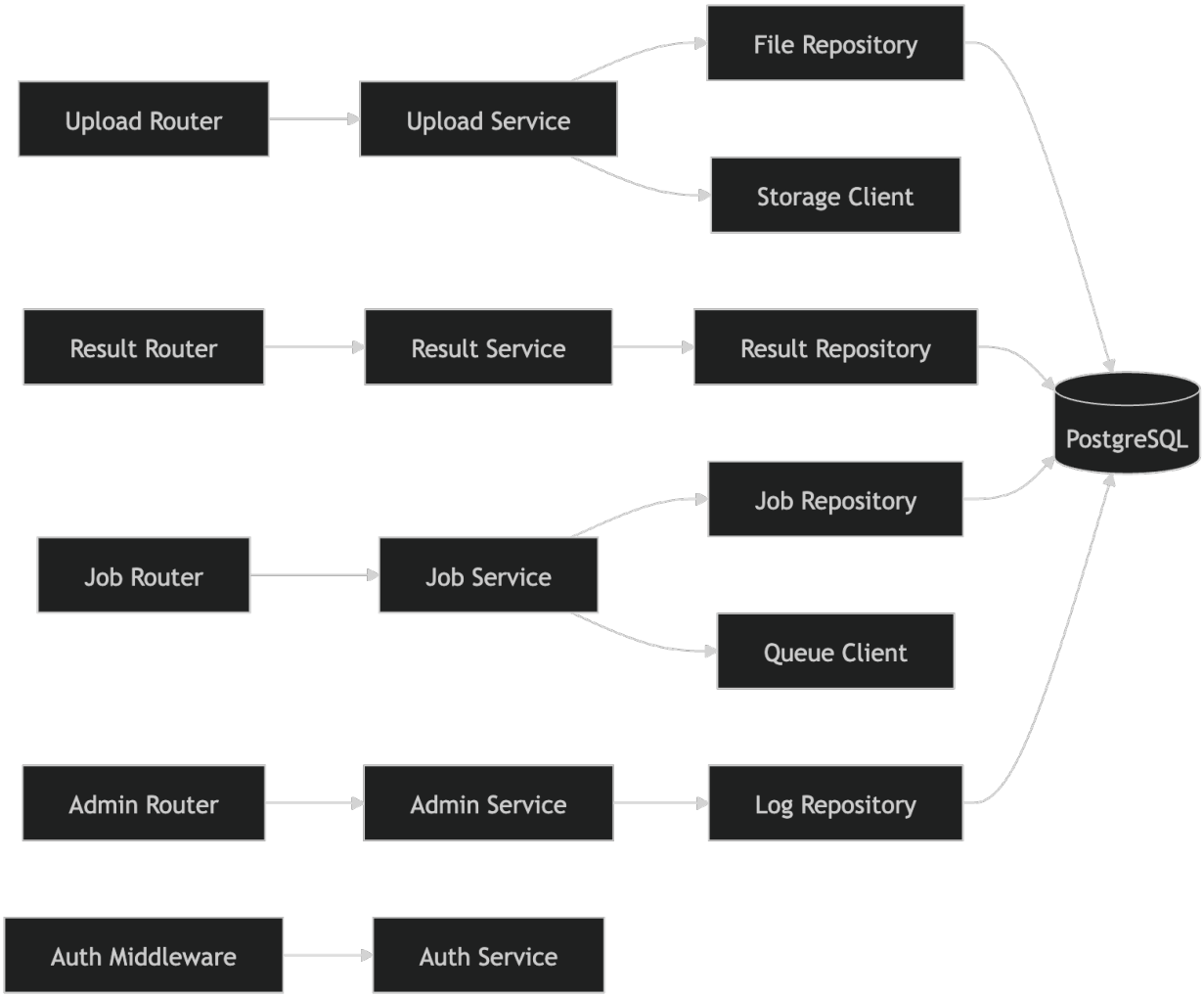
`auth_service`

Responsibilities:

- session validation
- access control

- user scoping

6.2 Backend Internal Flow Diagram



7. API LLD

7.1 Upload API

`POST /api/files/upload`

Request

- multipart form-data
- fields:
  - `file`
  - `page_selection_mode` optional
  - `page_numbers` optional

Response

```
{
  "fileId": "file_123",
  "fileName": "testset2.pdf",
  "status": "uploaded",
  "pageCount": 1
}
```

Validation

- only PDF
- size threshold
- checksum optional
- virus scan optional in production

7.2 Job Creation API

POST /api/jobs

Request

```
{
  "fileId": "file_123",
  "pageSelectionMode": "all",
  "pageNumbers": [],
  "outputFormats": ["png", "pdf", "json"]
}
```

Response

```
{
  "jobId": "job_456",
  "status": "queued"
}
```

7.3 Job Status API

GET /api/jobs/{jobId}

Response

```
{
  "jobId": "job_456",
  "fileId": "file_123",
  "status": "processing",
  "progress": 65,
  "step": "label_association",
  "createdAt": "2026-04-24T10:30:00Z",
  "updatedAt": "2026-04-24T10:31:10Z"
}
```

7.4 Result API

GET /api/jobs/{jobId}/result

Response


```
{
  "jobId": "job_456",
  "status": "completed",
  "previewUrl": "/files/output/job_456/annotated.png",
  "annotatedPdfUrl": "/files/output/job_456/annotated.pdf",
  "exports": {
    "json": "/files/output/job_456/segments.json",
    "csv": "/files/output/job_456/segments.csv"
  },
  "summary": {
    "pagesProcessed": 1,
    "segmentsDetected": 23,
    "segmentsMeasured": 19,
    "labelsMatched": 18
  }
}
```


7.5 Segments API

GET /api/jobs/{jobId}/segments

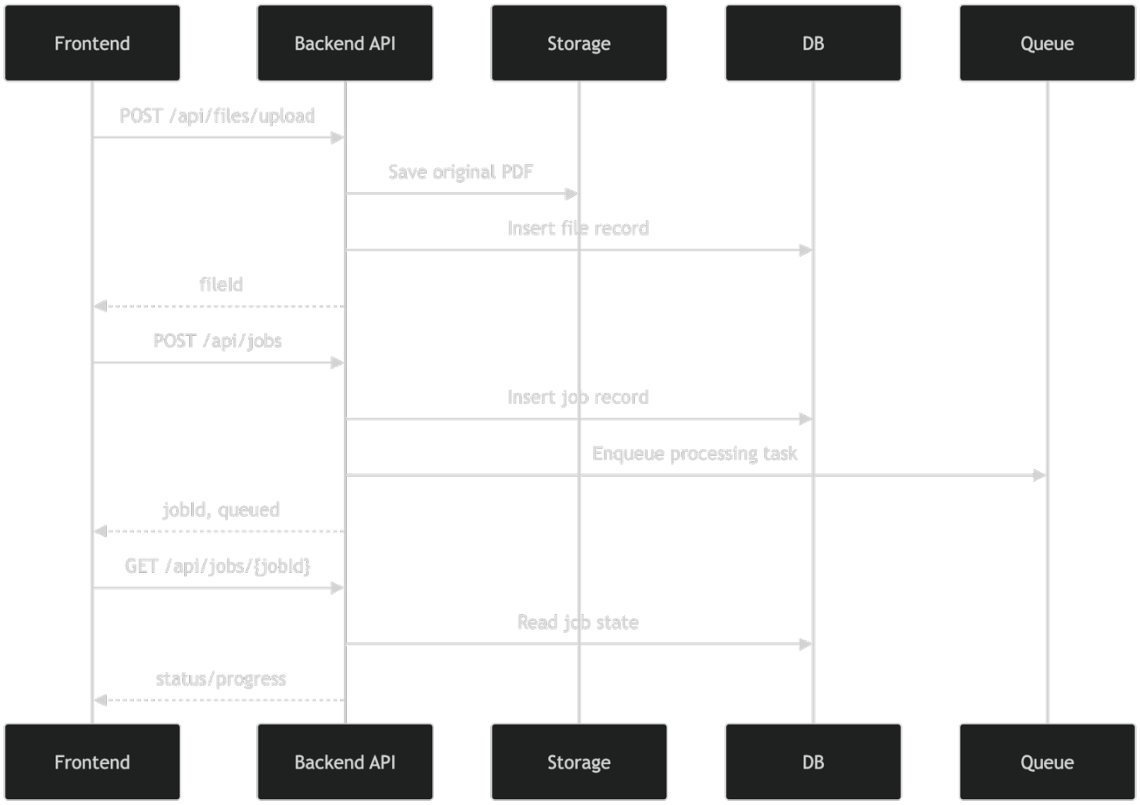
Response


```
{
  "segments": [
    {
      "segmentId": "D-001",
      "pageNumber": 1,
      "type": "supply",
      "shape": "rectangular",
      "sizeText": "12x8",
      "lengthFt": 14.5,
      "confidence": 0.91,
      "bbox": [100, 120, 300, 150]
    }
  ]
}
```


7.6 Admin APIs

- GET /api/admin/jobs
- GET /api/admin/jobs/{jobId}/logs
- GET /api/admin/metrics

8. API Sequence LLD



9. Database LLD

9.1 Core Tables

users

- id
- email
- name
- role
- created_at

files

- id
- user_id
- original_name
- storage_key
- mime_type
- size_bytes
- checksum
- page_count
- created_at

jobs

- id
- user_id
- file_id
- status
- progress_percent
- current_step
- error_code
- error_message
- created_at
- updated_at
- completed_at

job_pages

- id
- job_id
- page_number
- status
- rendered_image_key
- plan_crop_bbox
- scale_text
- scale_factor
- created_at

duct_segments

- id
- job_id
- page_number
- segment_code
- segment_type
- shape
- size_text
- width_value
- height_value
- diameter_value
- length_value
- length_unit
- confidence_score
- bbox_json
- polyline_json
- created_at

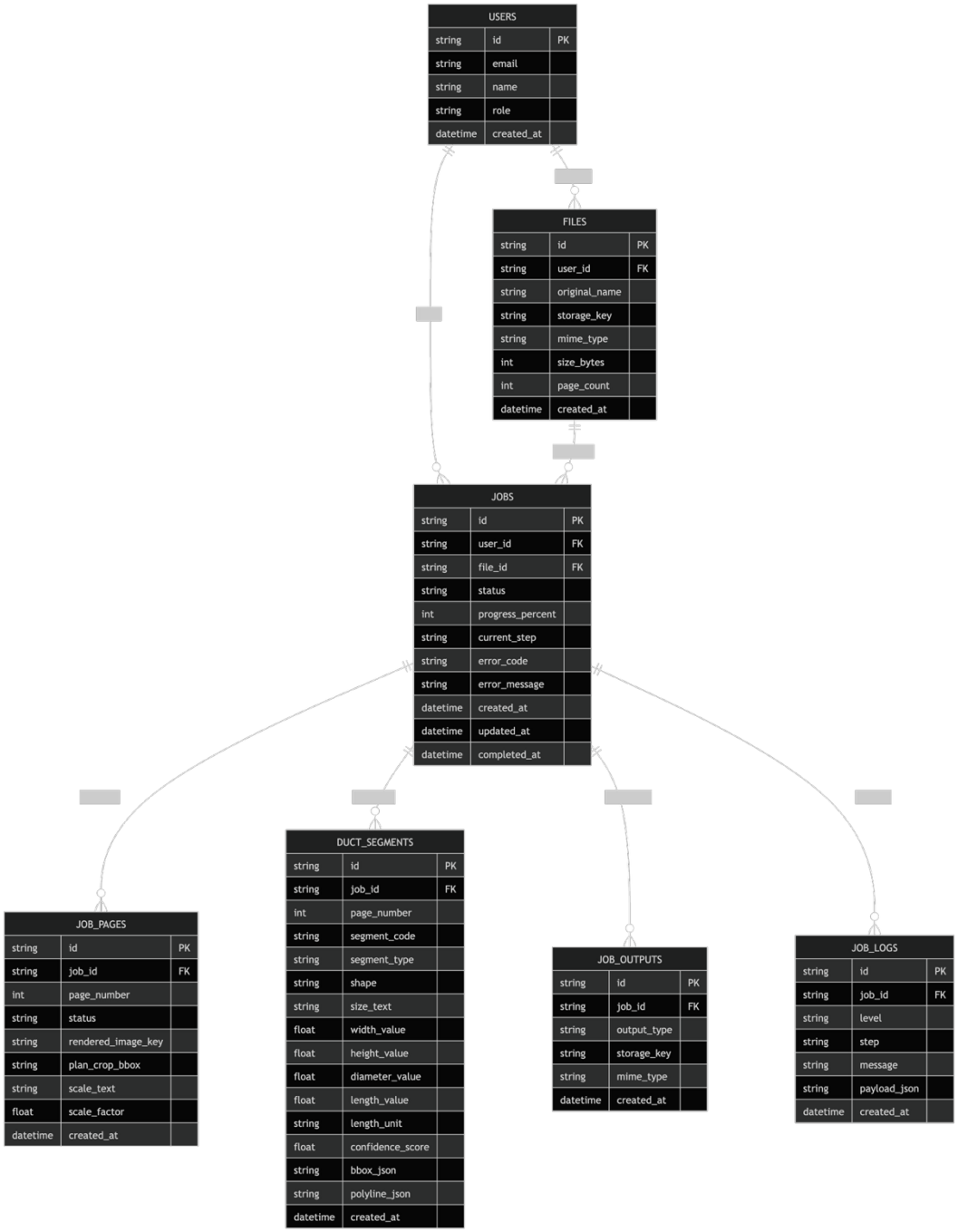
job_outputs

- id
- job_id
- output_type
- storage_key
- mime_type
- created_at

job_logs

- id
- job_id
- level
- step
- message
- payload_json
- created_at

9.2 ER Diagram



10. Queue and Worker LLD

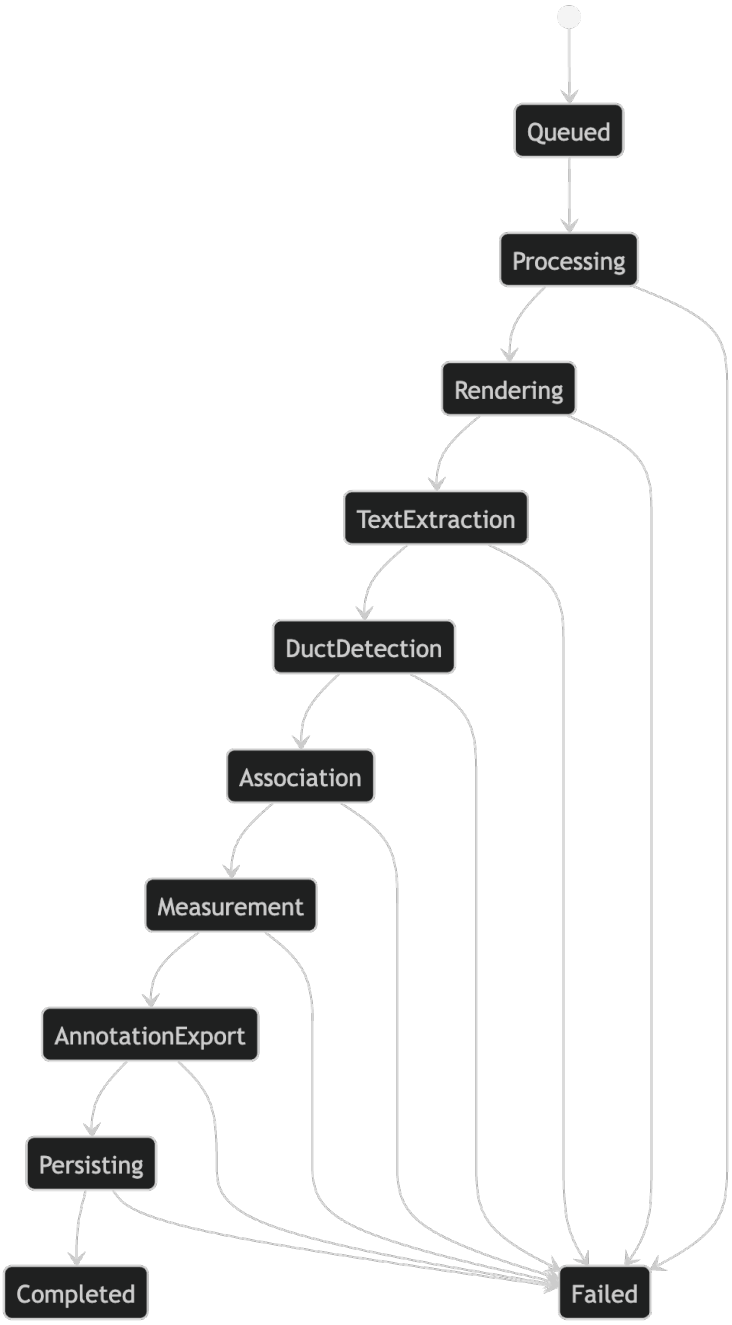
10.1 Queue Payload

```
{
  "jobId": "job_456",
  "fileId": "file_123",
  "userId": "user_789",
  "storageKey": "inputs/2026/04/testset2.pdf",
  "pageSelectionMode": "all",
  "pageNumbers": [],
  "outputFormats": ["png", "pdf", "json"],
  "requestedAt": "2026-04-24T10:30:00Z"
}
```

10.2 Worker Lifecycle

1. dequeue job
2. mark job as processing
3. download PDF from storage
4. iterate pages
5. run processing pipeline
6. store outputs
7. persist segments/metadata
8. mark job completed
9. on failure, mark failed and store logs

10.3 Worker State Diagram



11. Processing Engine LLD

This is the most important section.

11.1 Pipeline Modules

A. `pdf_loader.py`

Responsibilities:

- open PDF
- validate file readability
- count pages
- extract page dimensions

B. `renderer.py`

Responsibilities:

- render page at configurable DPI
- create raster image
- preserve transform mapping between PDF coordinates and image coordinates

C. `plan_region_detector.py`

Responsibilities:

- detect main plan drawing area
- suppress title block, notes, legends
- compute crop box

D. `text_extractor.py`

Responsibilities:

- extract words/phrases with coordinates
- merge nearby text tokens
- keep source coordinates

E. `dimension_parser.py`

Responsibilities:

- regex-based parsing of duct sizes
- parse rectangular and circular labels
- normalize units

F. `duct_detector.py`

Responsibilities:

- detect likely duct geometry from rendered plan
- find corridors/runs
- detect branches and elbows
- create raw duct candidates

G. `segment_builder.py`

Responsibilities:

- merge and split duct candidates
- produce duct segments
- assign local IDs

H. `association_engine.py`

Responsibilities:

- score candidate text labels against segments
- choose best label match
- attach confidence score

I. `scale_detector.py`

Responsibilities:

- detect explicit drawing scale text
- compute pixel/image to real-world conversion
- fallback to manual or configured scale

J. `measurement_engine.py`

Responsibilities:

- compute segment lengths
- apply unit conversion
- round output values

K. `annotation_renderer.py`

Responsibilities:

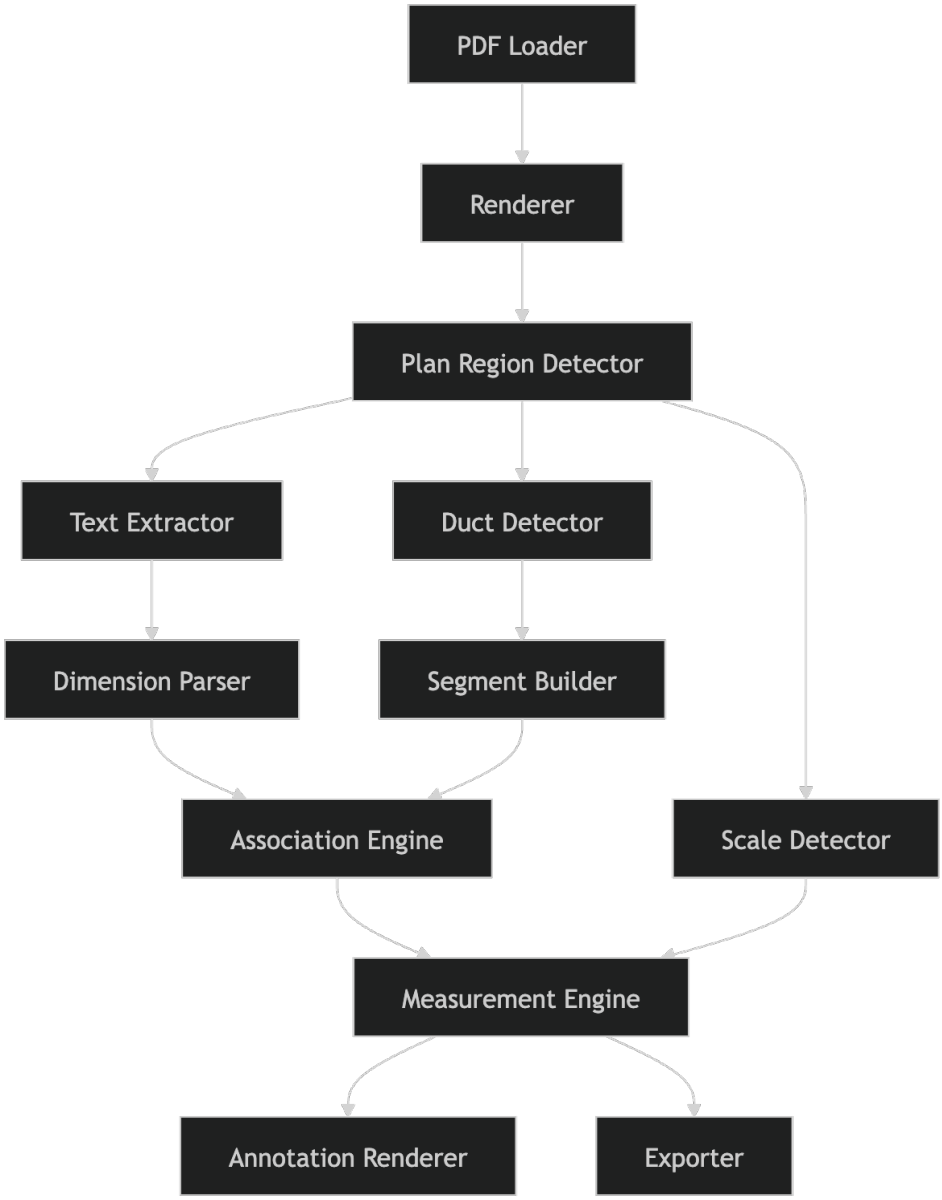
- draw highlighted ducts
- draw labels and IDs
- create final annotated page image/PDF

L. `exporter.py`

Responsibilities:

- export JSON/CSV
- write summaries
- produce manifest metadata

11.2 Processing Module Diagram



12. Internal Data Contracts

12.1 Page Context

```
{
  "pageNumber": 1,
  "pdfWidth": 1440,
  "pdfHeight": 900,
  "renderedWidth": 2880,
  "renderedHeight": 1800,
  "dpi": 300,
  "transform": {
    "scaleX": 2.0,
    "scaleY": 2.0
  }
}
```

12.2 Text Block

```
{
  "text": "12x8",
  "bbox": [210, 430, 260, 455],
  "pageNumber": 1,
  "source": "pdf_text",
  "rotation": 0
}
```

12.3 Duct Candidate

```
{
  "candidateId": "cand_01",
  "pageNumber": 1,
  "polyline": [[100, 100], [250, 100], [250, 180]],
  "bbox": [100, 90, 260, 190],
  "orientation": "mixed",
  "rawConfidence": 0.84
}
```

12.4 Segment Model

```
{
  "segmentId": "D-001",
  "pageNumber": 1,
  "polyline": [[100, 100], [250, 100]],
  "bbox": [100, 90, 250, 110],
  "shape": "rectangular",
  "type": "supply",
  "sizeText": "12x8",
  "lengthValue": 14.5,
  "lengthUnit": "ft",
  "confidence": 0.91
}
```

13. Label Association Logic

13.1 Inputs

- segment geometry
- nearby text blocks
- dimension-parsed text blocks

13.2 Scoring Factors

- distance from segment centerline
- parallel orientation
- HVAC size pattern confidence
- collision with unrelated geometry
- text placement in duct corridor
- symbol proximity if present

13.3 Output

- best matched label per segment
- confidence score
- unmatched labels list

13.4 Association Flow Diagram

14. Measurement Engine LLD

14.1 Measurement Logic

The system measures along the segment polyline.

Formula:

- pixel length from polyline
- convert to drawing units using page transform
- convert drawing units to real units using scale factor
- round using configured precision

14.2 Measurement Modes

- `centerline`
- `straight_run`
- `annotation_override`

Default:

- `centerline`

14.3 Rounding Rules

- feet: 2 decimals or feet-inch string
- metric: 1 or 2 decimals

15. Annotation Renderer LLD

15.1 Responsibilities

- draw duct paths
- draw outline/highlight
- draw label boxes
- avoid label collisions where possible
- preserve readability

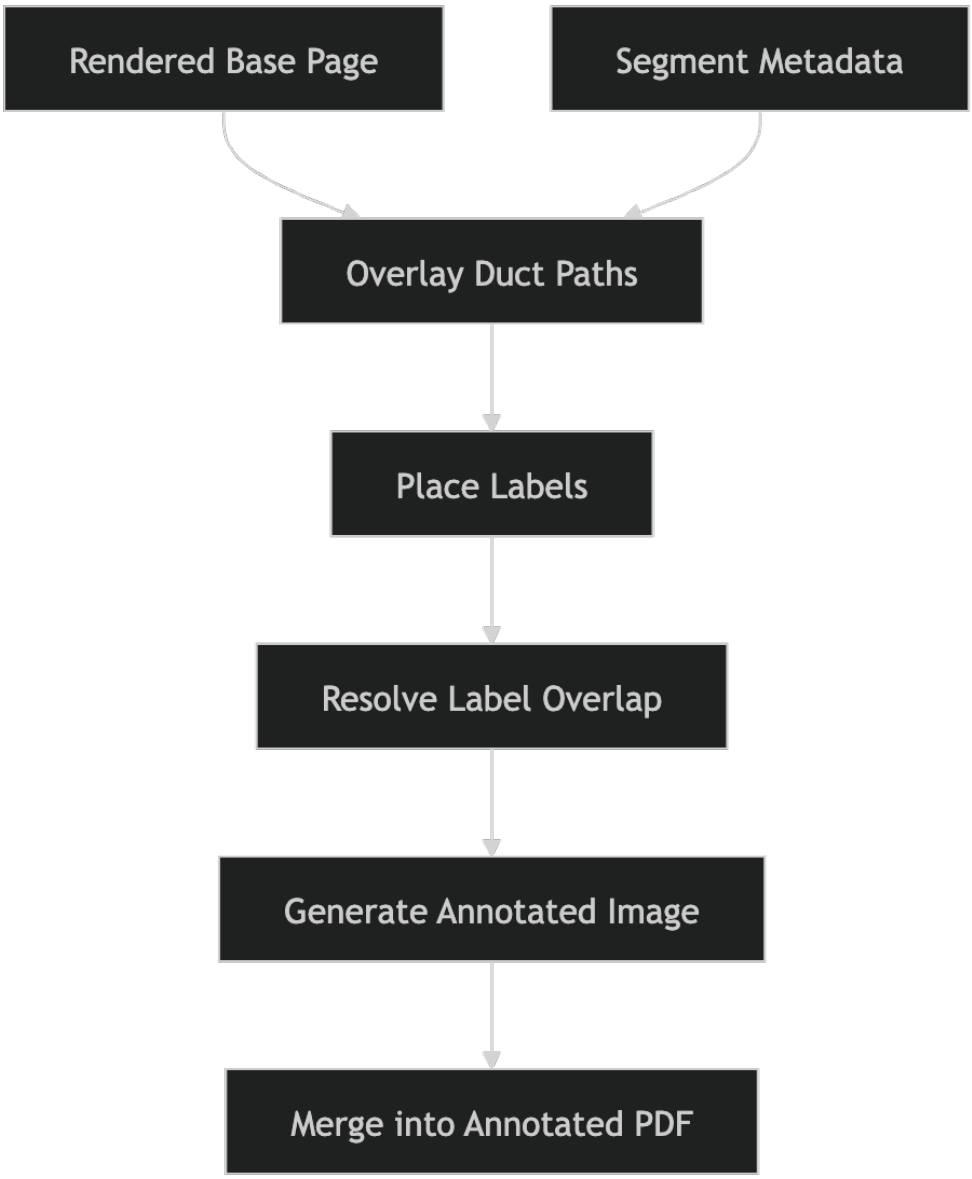
15.2 Output Artifacts

- annotated PNG per page
- annotated PDF full document
- optional debug images

15.3 Styling Rules

- supply ducts: blue
- return ducts: orange
- uncertain detections: dashed red
- label text: dark text on white box

15.4 Renderer Flow



16. Storage Key Conventions

Input

```
inputs/{year}/{month}/{fileId}/original.pdf
```

Intermediate

```
jobs/{jobId}/pages/{pageNumber}/rendered.png
jobs/{jobId}/pages/{pageNumber}/plan_crop.png
jobs/{jobId}/pages/{pageNumber}/debug_overlay.png
```

Outputs

```
jobs/{jobId}/outputs/annotated.png
jobs/{jobId}/outputs/annotated.pdf
jobs/{jobId}/outputs/segments.json
jobs/{jobId}/outputs/segments.csv
jobs/{jobId}/outputs/manifest.json
```

17. Suggested Code Structure

```
backend/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── upload_routes.py
│   │   ├── job_routes.py
│   │   ├── result_routes.py
│   │   └── admin_routes.py
│   ├── core/
│   │   ├── config.py
│   │   ├── auth.py
│   │   └── logging.py
│   ├── services/
│   │   ├── upload_service.py
│   │   ├── job_service.py
│   │   ├── result_service.py
│   │   └── admin_service.py
│   ├── repositories/
│   │   ├── file_repository.py
│   │   ├── job_repository.py
│   │   ├── segment_repository.py
│   │   ├── output_repository.py
│   │   └── log_repository.py
│   ├── workers/
│   │   ├── queue_client.py
│   │   ├── task_runner.py
│   │   └── job_processor.py
│   ├── processing/
│   │   ├── pdf_loader.py
│   │   ├── renderer.py
│   │   ├── plan_region_detector.py
│   │   ├── text_extractor.py
│   │   ├── dimension_parser.py
│   │   ├── duct_detector.py
│   │   ├── segment_builder.py
│   │   ├── association_engine.py
│   │   ├── scale_detector.py
│   │   ├── measurement_engine.py
│   │   ├── annotation_renderer.py
│   │   └── exporter.py
│   ├── models/
│   │   ├── file.py
│   │   ├── job.py
│   │   ├── job_page.py
│   │   ├── duct_segment.py
│   │   ├── job_output.py
│   │   └── job_log.py
│   └── schemas/
```

```
│        ├── upload.py
│        ├── job.py
│        ├── result.py
│        └── segment.py
frontend/
├── src/
│    ├── pages/
│    ├── components/
│    ├── hooks/
│    ├── services/
│    └── types/
```

18. Error Handling LLD

18.1 API Errors

Standard shape:

```
{
  "error": {
    "code": "INVALID_FILE_TYPE",
    "message": "Only PDF files are supported."
  }
}
```

18.2 Job Failure Categories

- invalid_pdf
- password_protected_pdf
- render_failed
- text_extraction_failed
- duct_detection_failed
- storage_write_failed
- unexpected_internal_error

18.3 Retry Policy

- retry storage/network failures
- do not retry malformed PDFs automatically
- record per-stage failure logs

19. Security and Access Control LLD

Rules

- user can access only own files/jobs/results
- admin can access all jobs/logs
- signed URLs for download where applicable
- file upload size limits
- MIME and extension validation
- sanitize log payloads

20. Config Model

Example configurable values:

```
app:
  max_upload_size_mb: 100
  default_output_formats: ["png", "pdf", "json"]

processing:
  render_dpi: 300
  measurement_mode: "centerline"
  min_segment_length_px: 15
  low_confidence_threshold: 0.5

annotation:
  show_segment_ids: true
  show_length: true
  show_size: true

storage:
  signed_url_expiry_minutes: 60
```

21. End-to-End LLD Flow

22. Implementation Priorities

Phase 1

- upload
- create job
- render page
- detect plan region
- detect ducts
- draw visual overlay
- return annotated PNG

Phase 2

- text extraction
- dimension parsing
- label association
- structured JSON output

Phase 3

- PDF export
- segment sidebar
- admin monitoring
- confidence/debug artifacts

Phase 4

- manual correction UI
- advanced shape support
- multi-user workspace features

23. LLD Summary

This LLD defines the implementable design for a **web-based asynchronous HVAC duct annotation platform** with:

- browser upload and result viewing
- API-driven job orchestration
- queue-based worker processing
- modular PDF/CV pipeline
- structured database persistence
- output artifact generation
- admin and failure observability

It is designed to match the actual problem behavior: a PDF input and a visually annotated duct-highlighted output, with optional structured metadata.